Rappi collection CHEAT SHEET

macOS / zsh

Delivery points and full country lists in. A new country workbook out.

Examples use Colombia. Country codes: ar Argentina / co Colombia / pe Peru / mx Mexico

1 Set up

Need Git, Python 3.12 and repository access.

```
git clone \
  https://github.com/YuruHuang/rappi-price-data.git
cd rappi-price-data
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Already cloned? Enter the project folder, check **git status**, preserve local edits, then **git pull --ff-only**. In later sessions, activate **.venv** again. Scrapy is not needed.

2 Edit delivery points

```
mkdir -p inputs
cp -n \
  config/examples/locations_multi_country.csv \
  inputs/locations.csv
open -e inputs/locations.csv
```

Replace the example points with your own. Keep the header; add or delete whole rows. **No fixed point count.**

Column	Meaning
country_code	ar, co, pe or mx
slug	Your stable, unique point ID
address	Full written address
lat / lng	Checked latitude / longitude
city / district	Optional context

Coordinates determine where Rappi searches. The address is a label; the code does not look it up or verify it. CSV, TSV, XLSX, JSON and YAML work.

One delivery point per row

country_code	slug	lat	lng
co	centro-01	4.598060	-74.076090

Excerpt; the full address is also required.

3 Keep or edit terms

Copy the **full original lists**. They include general, country and brand queries.

```
cp -n \
  config/examples/terms/co_tobacco.csv \
  inputs/co_tobacco.csv
cp -n \
  config/examples/terms/co_beverages.csv \
  inputs/co_beverages.csv
open -e inputs/co_tobacco.csv
open -e inputs/co_beverages.csv
```

Excerpt of your beverage copy

term
gaseosa
refresco
jugo
limonada

Add a query

To remove one, delete its row.

Keep the **term** header. One query per row. Save after editing.

Your file replaces the entire topic list. Keep every original query you still want. Leave the copies unchanged to keep the original plan.

Omit term-file options to use built-in defaults. cp -n preserves existing copies, including earlier edits. A new search term does not add a classification rule.

Original queries per delivery point

Country	Tobacco	Beverages
Argentina	27	45
Colombia	32	44
Peru	30	44
Mexico	32	45

4 Preview the plan

Both topics, your locations and your term copies:

```
python -m rappi_co.run \
  --country co --scope both \
  --locations-file inputs/locations.csv \
  --tobacco-terms-file \
    inputs/co_tobacco.csv \
  --beverage-terms-file \
    inputs/co_beverages.csv \
  --out data/updates/co_run_01 \
  --dry-run
```

Check before collecting

- ☐ Country and selected delivery points
- ☐ Every tobacco and beverage query
- ☐ planned_search_requests

Unchanged Colombia example: 32 tobacco + 44 beverage queries at each point. The preview makes no requests to Rappi.

2 points x **76** queries per point = **152** searches

5 Collect

Use the **same command**. Change the last option:

`--dry-run` → `--collect`

Use a **new output folder** for each new collection; the program creates it. Keep the input files unchanged while it runs.

Searches run one at a time, with random 1-3 second delays. Collection stops on an access challenge. Retries may add requests.

```
--min-delay 3 --max-delay 6
```

Optional slower pacing. Include the same settings if resuming.

Rappi collection OPTIONS AND OUTPUTS

macOS / zsh

One topic, several countries, and the files to open after collection.

Run every command from the project folder with .venv activated.

Choose one topic

Beverages with your full copied list

```
python -m rappi_co.run \
  --country co --scope beverages \
  --locations-file inputs/locations.csv \
  --terms-file inputs/co_beverages.csv \
  --out data/updates/co_beverages_01 \
  --dry-run
```

Tobacco with your full copied list

```
python -m rappi_co.run \
  --country co --scope tobacco \
  --locations-file inputs/locations.csv \
  --terms-file inputs/co_tobacco.csv \
  --out data/updates/co_tobacco_01 \
  --dry-run
```

Review, then replace --dry-run with --collect. Omit --terms-file to keep the built-in original list.

Use another country

Code	Country
ar	Argentina
co	Colombia
pe	Peru
mx	Mexico

Change **--country**, the country code in the term-file paths, and the output name. Use checked points tagged for that country.

Keep **rappi_co** as written. The package name stays the same for every country.

One topic uses --terms-file. Both topics use separate --tobacco-terms-file and --beverage-terms-file options, as on the front.

Run several countries

Tag **every location row** with country_code. Each selected country needs at least one matching point.

```
python -m rappi_co.release_update \
  --country co --country mx \
  --locations-file inputs/locations.csv \
  --out data/updates/co_mx_run_01 \
  --dry-run
```

country_code	
co	Plaza
co	Andino
mx	Bellas Artes
mx	Perisur



The batch collects **both topics** with each country's **own original terms**. With unchanged example points: Colombia 152 + Mexico 154 = 306 searches.

Review, then replace --dry-run with --collect. Omit all --country options for all four countries, with matching tagged points. All input rows are validated.

Different edited lists for each team

Use **rappi_co.run once per country**, with separate output folders. Run the front-page command for Colombia. Copy the full Mexico term lists, edit those copies, then use:

```
python -m rappi_co.run \
  --country mx --scope both \
  --locations-file inputs/locations.csv \
  --tobacco-terms-file \
    inputs/mx_tobacco.csv \
  --beverage-terms-file \
    inputs/mx_beverages.csv \
  --out data/updates/mx_run_01 \
  --dry-run
```

Copy mx_tobacco.csv and mx_beverages.csv as on the front, replacing co in the term filenames. release_update has no scope or term-file options. For one topic across countries, use one run command per country.

Open the workbook

```
open \
  data/updates/co_run_01/country_products
```

Open the country file after collection succeeds. Formatting is automatic.

Scope	Colombia filename
both	colombia.xlsx
beverages	co_beverages.xlsx
tobacco	co_tobacco.xlsx

- **Technical specifications**
- **Tobacco products**
- **SSBs**
- **Water**
- **Alcohol**

Both: all five tabs. Beverages: SSBs, Water, Alcohol. Tobacco: Technical specifications and Tobacco products. Other combined files: argentina.xlsx, peru.xlsx, mexico.xlsx.

Read the categories

Water	Plain drinking water, still or sparkling.
Alcohol	Recognized beer, wine, spirits or alcoholic cocktails. Alcohol-free variants are checked separately.
SSBs	Confirmed SSBs plus eligible candidates whose sweetening is unconfirmed.

Confirmed SSBs need caloric sweetening evidence. Total sugar alone does not prove added sugar. Product evidence, not the search term, determines the category.

Keep the whole run folder

Keep inputs, manifest and source data with the workbook. Interrupted both-topic run: repeat the original --collect command and add --resume with unchanged inputs. Single-topic: use a new output folder.